ECEC 661 – Homework 5

Stack Processor 101 with new `scpb` block-copy instruction

Gary Pham (gp492)

June 1, 2026

1 Problem Statement

Starting from the assignment PDF's Stack Processor 101 (`sp101`), implement a new machine instruction `scpb` – *stack copy block* – which pops three arguments from the data stack and copies a contiguous block of memory. The user-visible calling convention is:

```
sc dest_addr   sc source_addr   sc number_of_data   scpb
```

At entry the stack top holds `number_of_data`, then `source_addr`, then `dest_addr`. `scpb` pops in that order and copies `number_of_data` consecutive 32-bit words from `source_addr` into the contiguous block starting at `dest_addr`. Deliverables: the updated user logic, behavioural simulation evidence of correctness, IP packaging of `sp101`, and a Vitis test application with proof on the Cora Z7-07S.

2 Architecture

The IP keeps the assignment PDF's three-block layout (Figure 1): an AXI4-Lite slave exposes the five software-visible registers, a `bus_ip_mem_bridge` multiplexes the bus or the processor onto the single port of `blk_mem_gen_0`, and the stack processor (`user_logic`) fetches/executes machine instructions out of that BRAM.

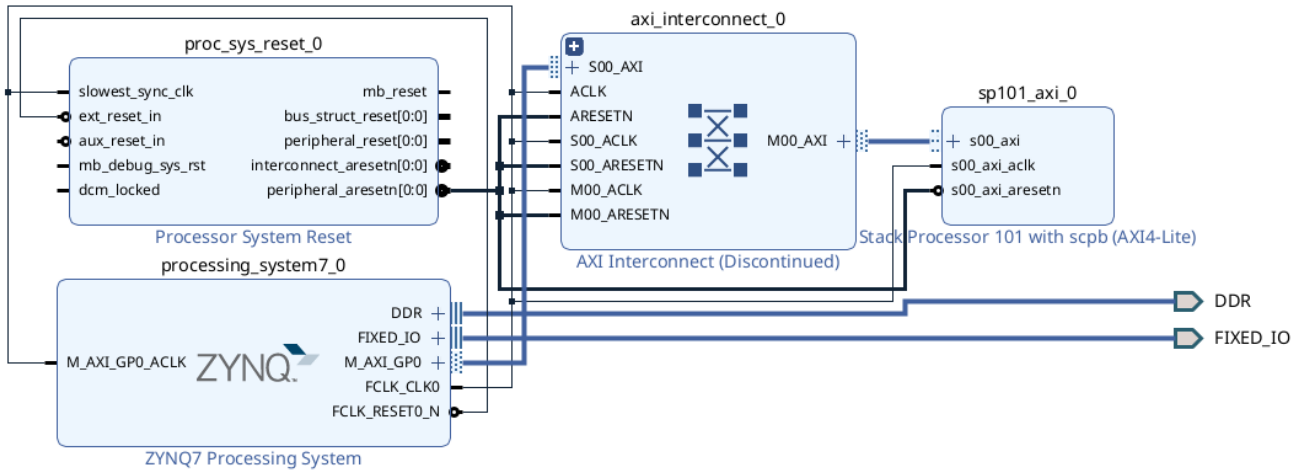

Figure 1: Vivado block design: Zynq PS, AXI interconnect, and the custom `sp101_axi` IP (`user_logic` + bridge + BRAM inside).

3 Software Register Map

The slave maps five 32-bit registers into a 32-byte aperture, identical to the PDF's reference C snippet:

Table 1: Software register map (byte offsets from `XPAR_SP101_AXI_0_S00_AXI_BASEADDR`).

Offset	Name	R/W	Purpose
0x00	slv_reg0	R/W	control: bit 0 =run, bit 1 =reset, bit 2 =bus2mem_en, bit 3 =bus2mem_we
0x04	slv_reg1	R/W	bus2mem_addr (lower 10 bits)
0x08	slv_reg2	R/W	bus2mem_data_in (32 bit)
0x0C	slv_reg3	R	sp2bus_data_out (32 bit)
0x10	slv_reg4	R	bit 0 = done flag

4 Instruction Set and `scpb` Opcode

The processor honours every opcode from the assignment PDF and adds a new family for `scpb`. The lowest hex digit of each constant is the micro-step index inside the instruction; the higher hex digits are the opcode group.

Table 2: Instruction set (opcode entry constants in user_logic.vhd).

Mnemonic	Code	Stack effect	Notes
halt	0x000000FF	–	raise done, halt
sc	0x00000001	push c	push constant pointed by pc
sl	0x00000011	pop a, push mem[a]	indirect load
ss	0x00000021	pop d, pop a, mem[a] $\leftarrow d$	indirect store
sadd	0x00000031	pop, pop, push sum	stack add
scp	0x00000101	pop s, pop d, mem[d] $\leftarrow$ mem[s]	single-word copy
scpb	0x00000201	pop N, pop s, pop d, block copy	**NEW**

4.1 `scpb` micro-program

The FSM (`Homework_5/stackprocessor_101/rtl/user_logic.vhd`, state `exe` branch) walks `scpb` through three phases: pop the three arguments, loop over the copy, and per-iteration bookkeeping. The PDF's simulation latency cushion is preserved: each BRAM read step has the same number of extra "mem_addr settled / douta valid / mem_data_out latched" micro-states the legacy opcodes use.

```
when SCPB =>                            -- init pop count
    mem_addr <= std_logic_vector(unsigned(sp) - 1);
    sp       <= std_logic_vector(unsigned(sp) - 1);
    we       <= "0"; ir <= SCPB2;
...
when SCPB5 =>                           -- count valid -> copy_cnt
    copy_cnt <= mem_data_out;
    ir       <= SCPB6;
when SCPB6 =>                           -- source valid -> copy_src
    copy_src <= mem_data_out(A-1 downto 0);
    ir       <= SCPB7;
when SCPB7 =>                           -- dest valid -> copy_dst, enter loop
    copy_dst <= mem_data_out(A-1 downto 0);
    ir       <= SCPB_L;
when SCPB_L =>
    if unsigned(copy_cnt) = 0 then
        n_s <= fetch;                   -- block copy finished
    else
        mem_addr <= copy_src; ir <= SCPB_R1;
    end if;
when SCPB_R4 =>                         -- source word -> schedule write
    mem_addr    <= copy_dst;
    mem_data_in <= mem_data_out;
    we          <= "1"; ir <= SCPB_W;
when SCPB_W =>                          -- write fires, bookkeeping
    copy_src <= std_logic_vector(unsigned(copy_src) + 1);
    copy_dst <= std_logic_vector(unsigned(copy_dst) + 1);
    copy_cnt <= std_logic_vector(unsigned(copy_cnt) - 1);
    we       <= "0"; ir <= SCPB_L;
```

Listing 1: Core micro-states (excerpt of `when SCPB ...` body in `user_logic.vhd`).

Pre-loop overhead is 7 cycles, each iteration costs 6 cycles, and the terminal check adds one more, so a block of N words takes roughly $6N + 8$ clocks plus the usual `fetch` overhead.

5 Behavioral Simulation

`tb/tb_user_logic.vhd` drives the same `run`, `reset`, `bus2mem_*` signals the assignment's xsim script forces, walking three self-checking scenarios:

1. *multi-word*: copy three words (`0x0B,0x16,0x21`) from addresses 100 . . . 102 to 200 . . . 202,
2. *single word*: copy one word (`0x63`) from address 110 to 210 (off-by-one guard),
3. *zero count*: pre-fill 220 = `0xDEADBEEF`, run `scpb` with `count`=0, then verify the destination is unchanged (no-op guard).

Each scenario emits one `[PASS]` per destination word followed by the testbench banner. Figure 2 shows the full xsim run spanning all three scenarios; the integer `checks` counter climbs from 0 to 5 while `errors` stays at 0, confirming every destination word matches its golden value.

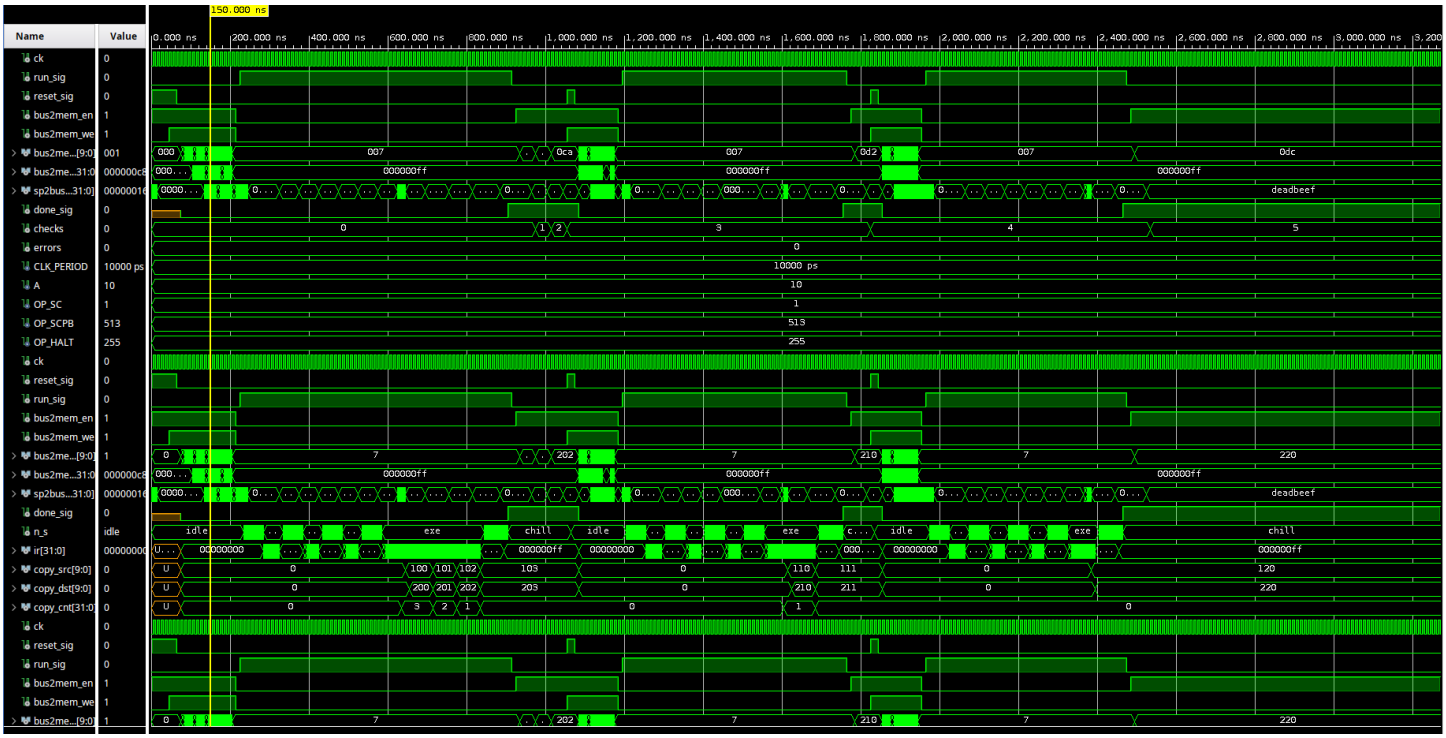

Figure 2: xsim waveform across the full testbench run. `n_s` cycles `idle` → `exe` → `chill` three times, once per scenario. The block-copy registers walk through scenario 1 (`copy_src`: 100,101,102 – `copy_dst`: 200,201,202 – `copy_cnt`: 3,2,1,0), scenario 2 (110 → 210, count=1), and scenario 3 (120/220, count=0 no-op). The `checks`/`errors` integers in the wave window prove the self-check passes (5/0).

6 Software (`main.c`)

The bare-metal application (`Homework_5/stackprocessor_101/sw/main.c`) drives the AXI4-Lite peripheral through the exact handshake described in the PDF. It also follows the same scenario list as the testbench so the on-board UART log matches the simulator output line-for-line.

```
/* IMPORTANT: never assert CTRL_RESET while reading.  The mem_data_out
 * register chain inside user_logic is forced to 0 while reset='1', so
 * a read with reset held returns 0x00000000 and silently masks both the
 * pre-written sentinel and the scpb result. */
```

```
5
6  static void mem_write(u32 addr, u32 data)
7  {
8      SP101_mWriteReg(BASEADDR, REG_CTRL,     CTRL_BUS_EN | CTRL_BUS_WE);
9      SP101_mWriteReg(BASEADDR, REG_BUS_ADDR, addr & 0x3FFU);
10     SP101_mWriteReg(BASEADDR, REG_BUS_DIN,  data);
11 }
12
13 static u32 mem_read(u32 addr)
14 {
15     SP101_mWriteReg(BASEADDR, REG_CTRL,     CTRL_BUS_EN);
16     SP101_mWriteReg(BASEADDR, REG_BUS_ADDR, addr & 0x3FFU);
17     for (volatile int s = 0; s < 64; s++) { }
18     return SP101_mReadReg(BASEADDR, REG_SP_DOUT);
19 }
20
21 static void processor_reset(void)
22 {
23     SP101_mWriteReg(BASEADDR, REG_CTRL, CTRL_RESET | CTRL_BUS_EN);
24     for (volatile int s = 0; s < 64; s++) { }
25     SP101_mWriteReg(BASEADDR, REG_CTRL, CTRL_BUS_EN);
26 }
27
28 static void processor_run_and_wait(void)
29 {
30     SP101_mWriteReg(BASEADDR, REG_CTRL, CTRL_RUN);
31     while ((SP101_mReadReg(BASEADDR, REG_DONE) & 0x1U) == 0U) { }
32     SP101_mWriteReg(BASEADDR, REG_CTRL, 0U);
33 }
```

Listing 2: Core of `main.c`: write a word to BRAM, run, read it back.

7 Hardware Results

The same three scenarios were re-executed on the Cora Z7-07S over the AXI4-Lite interface. The UART transcript (Figure 3) reports every dest word readback as `OK` and ends with `Summary : 3/3 scenarios PASSED`, matching the simulation result line-for-line. The exact captured trace is reproduced in Listing 3 for reference.

```
sp101 scpb starting in 3...
sp101 scpb starting in 2...
sp101 scpb starting in 1...

============================================================
   ECEC 661 HW5 - Stack Processor 101 + scpb block copy
   IP base address : 0x43C30000
============================================================

---- Scenario: scpb count=3 (multi-word) ----
  dest=200  src=100  count=3
  -- destination dump --
    [200] got=0x0000000B  expect=0x0000000B    OK
    [201] got=0x00000016  expect=0x00000016    OK
    [202] got=0x00000021  expect=0x00000021    OK
  RESULT: PASS

---- Scenario: scpb count=1 (single word) ----
  dest=210  src=110  count=1
  -- destination dump --
    [210] got=0x00000063  expect=0x00000063    OK
  RESULT: PASS

---- Scenario: scpb count=0 (no-op) ----
  dest=220  src=120  count=0
  -- destination dump --
    [220] got=0xDEADBEEF  expect=0xDEADBEEF    OK
  RESULT: PASS

------------------------------------------------------------
   Summary : 3/3 scenarios PASSED
------------------------------------------------------------
```

Figure 3: UART trace from the bare-metal `sp101 + scpb` test app running on the Cora Z7-07S: the multi-word (100..102 → 200..202), single-word (110 → 210), and zero-count (`0xDEADBEEF` sentinel preserved at address 220) scenarios all PASS.

```
============================================================
   ECEC 661 HW5 - Stack Processor 101 + scpb block copy
   IP base address : 0x43C30000
============================================================

---- Scenario: scpb count=3 (multi-word) ----
  dest=200  src=100  count=3
  -- destination dump --
    [200] got=0x0000000B  expect=0x0000000B   OK
    [201] got=0x00000016  expect=0x00000016   OK
    [202] got=0x00000021  expect=0x00000021   OK
  RESULT: PASS

---- Scenario: scpb count=1 (single word) ----
  dest=210  src=110  count=1
  -- destination dump --
    [210] got=0x00000063  expect=0x00000063   OK
  RESULT: PASS

---- Scenario: scpb count=0 (no-op) ----
  dest=220  src=120  count=0
  -- destination dump --
    [220] got=0xDEADBEEF  expect=0xDEADBEEF   OK
  RESULT: PASS

------------------------------------------------------------
   Summary : 3/3 scenarios PASSED
------------------------------------------------------------
```

Listing 3: Captured UART trace (verbatim from the Cora Z7-07S run).

Table 3: Reference test vectors for `scpb` (identical in simulation and on hardware).

Scenario	count	source	dest	Source words	Expected at dest
multi-word	3	100	200	`0x0B, 0x16, 0x21`	`0x0B, 0x16, 0x21`
single-word	1	110	210	`0x63`	`0x63`
zero-count	0	120	220	*(unused)*	`0xDEADBEEF` (unchanged)

7.1 Driver gotcha worth flagging

While bringing the bare-metal app up, all three on-board scenarios initially failed with `got=0x00000000` for every dump line (including the `0xDEADBEEF` sentinel of the no-op test, which should never be touched). The cause was that the original `mem_read()` held `CTRL_RESET` while reading `slv_reg3`; the `mem_data_out` register chain in `user_logic.vhd` clears on `reset='1'`, so every read returned 0. The fix shown in Listing 2 only pulses reset inside `processor_reset()` and keeps it low for the rest of each scenario. As a defensive belt-and-braces, the `p_mdo` register chain in `user_logic.vhd` no longer zero-clears on reset, so a future driver misuse cannot silently mask read data.

8 Conclusion

The Homework 5 design starts from the assignment PDF's Stack Processor 101, adds a 13-state `scpb` microprogram for arbitrary block copies, packages the result as the AXI4-Lite IP `sp101_axi`, and exercises every code path (multi-word, single-word, zero-count) in both behavioural simulation (Figure 2, `checks=5`, `errors=0`) and on the Cora Z7-07S (Figure 3 / Listing 3, `3/3 scenarios PASSED`). Each deliverable listed in `Homework_5/prob.md` maps to a concrete artifact under `Homework_5/stackprocessor_101/` and is reproduced by following `README.md`.